



工单 ID:690801

禅道 ID:

中化 k8s pvc 挂载 nfs 触发 vmcore 问题 分析报告

麒麟软件有限公司

2026 年 7 月 3 日

文档变更记录

版本号	变更说明	变更人	审核人	日期
V1.0	初步分析报告	杨进龙		2026 年 06 月 26 日
V1.1	补充分析过程	熊乐		2026 年 06 月 26 日
v1.2	完善分析过程	张国栋		2026 年 07 月 03 日

目录

一、 系统环境及配置	1
二、 问题现象描述	2
三、 问题分析过程	2
四、 问题分析结论	6
五、 问题解决方案	7

一、系统环境及配置

系统环境	*物理机/虚拟机/云/容器	物理机
宿主机 /*kos 为 guest 必填 */	*宿主机处理器	/
	*云平台信息	/
网络环境	*外网/私有网络/无网络	私有网络
硬件环境	*机型	裸金属
	*处理器	X86
	内存	未获取
	显卡	未获取
	主板型号	未获取
	整机类型/架构	x86
	固件版本	未获取
	系统硬盘	未获取
软件环境	*具体操作系统版本	Kylin-Server-10-SP1- Release-Build20- 20210518-x86_64
	*内核版本	4.19.90- 23.57.v2101.ky10.x86_64

	*桌面系统 buildid (见/etc/kylin-build)	/
第三方软件	*有/无, 如有已安装的第三方软件、 第三方平台、客户自研程序列出具 体名称、版本号	无

二、问题现象描述

k8s 环境,应用容器会使用 pvc 挂载 nfs,使用 Ubuntu 的基础镜像没问题,
换成 kylin 的基础镜像在应用重启时触发 vmcore。

重启时间 2026-06-26 14:45

```
[ 1928.886924] CPU: 84 PID: 52027 Comm: umount.nfs Kdump: loaded Tainted: G          OE      4.19.90-23.57.v2101.ky10.x86_64 #1
[ 1928.899930] Hardware name: Suma R6240H0/62DB32, BIOS CXYH051029 09/06/2023
[ 1928.908096] RIP: 0010:kfree+0x4f/0x160
[ 1928.912764] Code: 80 49 01 da 0f 82 1b 01 00 00 48 c7 c7 00 00 00 80 48 2b 3d 3b b6 0b 01 49 01 fa 49 c1 ea 0c 49 c1 e2 06 4c 03 15 19 b6 0b 01 <49> 8b 42 08 48 8d 50 ff a8 01 4c
[ 1928.934206] RSP: 0018:ffffb2bc8f2af678 EFLAGS: 00010203
[ 1928.940523] RAX: 000000000000001e RBX: 9db19651fd8d1b1d RCX: 0000000000000000
[ 1928.948973] RDX: 0000000000000000 RSI: ffff43200002fde0 RDI: 0000693200000000
[ 1928.957422] RBP: ffff96fd80bf7000 R08: 0000000000000000 R09: 0000000000000000
[ 1928.965871] R10: 0275dc4f51ee3440 R11: 0000000000000000 R12: ffffffff0ae363d
[ 1928.974321] R13: 0000000000000000 R14: ffff96fd6181ec58 R15: 0000000000000001
[ 1928.982769] FS: 00007fe8baa22840 (0000) GS: ffff96fdbfd00000 (0000) knlgs:0000000000000000
[ 1928.992284] CS: 0010 DS: 0000 ES: 0000 CR0: 0000000080050033
[ 1928.996183] CR2: 00007fe8baa22840 RAX: 0000000000000000 RDI: 0000693200000000
```

三、问题分析过程

Pod 应用重启, 在执行 umount.nfs (卸载 NFS 文件系统) 操作时触发内核崩溃, 崩溃发生在 kfree 函数, 崩溃点 kfree+0x4f, 正是在释放内存时访问了非法指针。

关键调用链:

umount

→ nfs_kill_super

→ generic_shutdown_super

→ evict_inodes

→ evict

```
→ nfs_evict_inode
→ truncate_inode_pages_range
→ delete_from_page_cache_batch
→ page_cache_free_page
→ nfs_readdir_clear_array
→ kfree (GPF)
```

从堆栈分析, 卸载 NFS 时超级块被销毁, 内核 evict 所有 inode。对于 NFS 目录 inode, 其页面缓存保存了 readdir 的结果数组, 当页面被从缓存中移除时, nfs_readdir_clear_array 负责释放数组内的条目标针, 此处调用 kfree 时访问了非法指针。对应内核代码:

```
C dir.c  ×
fs > nfs > C dir.c
182     kunmap_atomic(array),
183 }
184
185 /*
186  * we are freeing strings created by nfs_add_to_readdir_array()
187  */
188 static
189 void nfs_readdir_clear_array(struct page *page)
190 {
191     struct nfs_cache_array *array;
192     int i;
193
194     array = kmap_atomic(page);
195     for (i = 0; i < array->size; i++)
196         kfree(array->array[i].string.name);
197     nfs_readdir_array_init(array);
198     kunmap_atomic(array);
199 }
200
```

首先给 nfs_cache_array 添加一个固定值的字段 magic, 并且在 init_array 的时候赋值为 0x5A5AAA5。

```
struct nfs_cache_array {
    u64 last_cookie;
    int magic;
    unsigned int size;
    unsigned char page_full : 1,
                  page_is_eof : 1;
    struct nfs_cache_array_entry array[];
};
```

```
static void nfs_readdir_page_init_array(struct page *page, u64 last_cookie)
{
    struct nfs_cache_array *array;

    array = kmap_atomic(page);
    nfs_readdir_array_init(array);
    array->last_cookie = last_cookie;
    array->magic = 0x5A5AA5;
    kunmap_atomic(array);
}
```

在 `nfs_readdir_add_to_array` 和 `nfs_readdir_clear_array` 增加打印信息，打印 `kfree` 之前的 `array->size` 以及每次通过 `readdir` 增加 `array->size` 的大小和对应的 `array->magic` 值。

```
@@ -192,7 +194,8 @@ void nfs_readdir_clear_array(struct page *page)
    int i;

    array = kmap_atomic(page);
-    printk("array->size = %d\n", array->size);
+    printk("NFS: clear_array page=%p index=%lu size=%u, magic=%x\n",
+        page, page->index, array->size, array->magic);
    for (i = 0; i < array->size; i++) {
        printk("i = %d\n", i);
        kfree(array->array[i].string.name);
    }

@@ -274,6 +277,8 @@ int nfs_readdir_add_to_array(struct nfs_entry *entry, struct page *page)
    array->last_cookie = entry->cookie;
    array->size++;
    printk("array add %d\n", array->size);
+    printk("NFS: add_to_array page=%p size=%u\n",
+        page, array->size);
    if (entry->eof != 0)
        nfs_readdir_array_set_eof(array);
```

打印崩溃 `page` 的相关信息，可以看到 `size=1635021678`，这是一个异常值，根本不可能通过 `array->size++` 得到。并且这个 `page` 为 `00000000a2781d48` 在

readdir_add_to_array 函数一次都没出现。证明这个 page 是突然冒出来的，不是通过正常路径 nfs_readdir_clear_array 产生的。同时 magic 为一个乱的数据，跟之前初始化设置 0x5A5AAA5 不一致。

```
[ 343.759048] NFS: clear_array page=00000000a2781d48 index=0 size=1635021678, magic=692e7370
```

从上面分析得知这个不是通过 readdir_add_to_array 添加进来的，分析其他可能的来源途径可能是从 nfs_symlink 来。nfs_symlink 先 alloc_page 分配裸页，RPC 成功后再挂进 symlink inode 的 page cache。

RPC 成功

- 得到 symlink inode (d_inode(dentry))
- add_to_page_cache_lru(page, inode->i_mapping, 0, GFP_KERNEL)
- SetPageUptodate(page)
- unlock_page / put_page

为了证明来源途径是从 nfs_symlink 这里来，将 add_to_page_cache_lru 相关代码删除，意思是确认是 symlink inode 后，不把页加入到 symlink 的 page cache。这里的删除只会影响性能不会影响功能。删除之后发现系统不会死机，证明异常来源于 nfs_symlink。

```
@@ -1935,16 +1935,8 @@ int nfs_symlink(struct inode *dir, struct dentry *dentry, const char *synname)
 * No big deal if we can't add this page to the page cache here.
 * READLINK will get the missing page from the server if needed.
 */
if (!add_to_page_cache_lru(page, d_inode(dentry)->i_mapping, 0,
                           GFP_KERNEL)) {
    SetPageUptodate(page);
    unlock_page(page);
    /*
     * add_to_page_cache_lru() grabs an extra page refcount.
     * Drop it here to avoid leaking this page later.
     */
    put_page(page);
} else
+
+
    __free_page(page);
```

nfs_symlink 中从 page 分配到 add_to_page_cache_lru 过程中，对 page 进行

操作的行为只有 `error = NFS_PROTO(dir)->symlink(dir, dentry, page, pathlen, &attr);`这一行。因为是 nfsv3 挂载，调用链如下：

```
NFS_PROTO(dir)->symlink
→ nfs3_proc_symlink
→ nfs3_do_create
```

自定义一个 `nfs3_do_create_kylin` 函数进行相关信息的打印，打印输出「哪条 RPC、哪个 dentry、fh 多大、fattr 哪些字段有效、mode 八进制值、服务端 mode 是否算 symlink。

```
+static struct dentry *
+nfs3_do_create_kylin(struct inode *dir, struct dentry *dentry, struct nfs3_createdata *data)
+{
+    int status;
+
+    status = rpc_call_sync(NFS_CLIENT(dir), &data->msg, 0);
+    nfs_post_op_update_inode(dir, data->res.dir_attr);
+    if (status != 0)
+        return ERR_PTR(status);
+    printk("NFS: %s %pd server reply fh_size=%u fattr_valid=0x%llx type=%d mode=%d fileid=%d mode_value=%06o symlink_mode=%d\n",
+    data->msg.rpc_proc->p_name, dentry, data->res.fh->size,
+    (unsigned long long)data->res.fattr->valid,
+    !!((data->res.fattr->valid & NFS_ATTR_FATTR_TYPE),
+    !!((data->res.fattr->valid & NFS_ATTR_FATTR_MODE),
+    !!((data->res.fattr->valid & NFS_ATTR_FATTR_FILEID),
+    data->res.fattr->mode,
+    data->msg.rpc_proc == &nfs3_procedures[NFS3PROC_SYMLINK] &&
+    (data->res.fattr->valid & NFS_ATTR_FATTR_MODE) &&
+    S_ISLNK(data->res.fattr->mode));
+
+    return nfs_add_or_obtain(dentry, data->res.fh, data->res.fattr, NULL);
+}
+
+static void nfs3_free_createdata(struct nfs3_createdata *data)
+{
+    kfree(data);
+}
+
+@@ -532,7 +556,7 @@ nfs3_proc_symlink(struct inode *dir, struct dentry *dentry, struct page *page,
+data->arg.symlink.pathlen = len;
+data->arg.symlink.sattr = sattr;
+
+-    d_alias = nfs3_do_create(dir, dentry, data);
++    d_alias = nfs3_do_create_kylin(dir, dentry, data);
+status = PTR_ERR_OR_ZERO(d_alias);
```

分析打印出来的信息，返回了很多 `symlink_mode=0` 的信息，说明服务端返回的根本就不是 symlink。简单来说返回的不是软链接。

```
[四 7月 2 17:49:16 2026] NFS: SYMLINK i18n server reply fh_size=32 fattr_valid=0x27e7f type=1 mode=1 fileid=1 mode_value=040777 symlink_mode=0
[四 7月 2 17:49:16 2026] S_ISLNK page=ffffe82cc07617c0 inode=ffff8bf25c71e680 ino=12593 mode=40777 mapping=ffff8bf25c71e7f0 index=0
[四 7月 2 17:49:16 2026] NFS: symlink(0:276/12093, template, /data/apps.install._bpm.portal.template)

[四 7月 2 17:49:16 2026] NFS: SYMLINK icon96.png server reply fh_size=32 fattr_valid=0x27e7f type=1 mode=1 fileid=1 mode_value=100777 symlink_mode=0
[四 7月 2 17:49:16 2026] S_ISLNK page=ffffe82cc0af17c0 inode=ffff8bf2535bd0a0 ino=73326 mode=100777 mapping=ffff8bf2535bd210 index=0
[四 7月 2 17:49:16 2026] NFS: symlink(0:276/12093, i18n, /data/apps.install._bpm.portal.i18n)

[四 7月 2 17:49:16 2026] NFS: SYMLINK icon64.png server reply fh_size=32 fattr_valid=0x27e7f type=1 mode=1 fileid=1 mode_value=100777 symlink_mode=0
[四 7月 2 17:49:16 2026] S_ISLNK page=ffffe82cc0ae55c0 inode=ffff8bf2535b73a0 ino=73324 mode=100777 mapping=ffff8bf2535b7510 index=0
[四 7月 2 17:49:16 2026] NFS: symlink(0:276/12091, icon96.png, /data/apps.install._bpm.platform.icon96.png)
```

四、问题分析结论

客户端向服务端发送 `symlink` 操作后，服务端返回不是 `symlink`，返回的是

目录,导致客户端误把 symlink 页放进目录 page cache,进而导致客户端在 umount 时把 symlink 页按目录数组去 kfree , 导致宕机。

五、问题解决方案

1. 客户端向服务端发送 symlink 操作后,服务端为什么会返回不是 symlink, 这是问题产生根因。
2. 客户端进行过滤, 当服务端返回的不是 symlink 后, 不再加入 page cache, 这个只是规避方案。